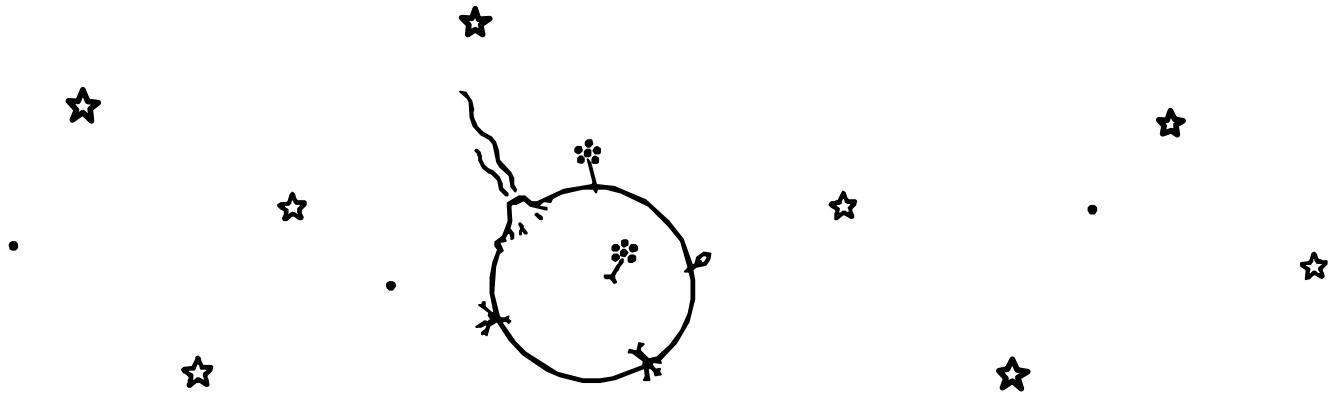


Planetaire Mono

B612 LETTERFORMS
HACK INFRASTRUCTURE
NERD FONT ICONS

A beautiful, highly legible monospace font
for terminals, editors, and agentic work

Version 0.1.5 · 2026-09-07
github.com/jlevy/planetaire

Planetaire Mono assembled and maintained by **Joshua Levy**
B612 Mono letterforms by **Intactile Design** for **Airbus**
Hack base punctuation, symbols, and metrics by **Chris Simpkins**
Nerd Fonts 10,000+ icons by **Ryan McIntyre**
Dotted zero inspired by the B612 fork by **Carlos Eduardo de Paula**

ABOUT B612

B612 began not as a typeface but as an aviation research program. In 2010, Airbus, ENAC (the French civil aviation university), and the Université de Toulouse III set out to define and validate an “aeronautical font” for cockpit screens: text a pilot can read correctly while fatigued, at oblique angles, or under vibration, glare, or near-darkness.

The shapes were derived experimentally before they were drawn. Jean-Luc Vinot (ENAC) and Sylvie Athènes (Toulouse III) built confusion matrices of when and how characters get misread (“Legible, are you sure?” at CHI 2012). In their controlled study, the prototype that became B612 drew slightly more correct reads than Verdana and clearly outperformed the legacy avionics font. Airbus then commissioned the Montpellier interface studio Intactile Design (Nicolas Chauveau, Thomas Paillot, and Jonathan Favre-Lamarine) to draw the full family of eight variants.

B612 is named for the asteroid home of the Little Prince by Antoine de Saint-Exupéry, who was himself an aviator.

B612’s unusual character is a humanist answer to an instrument-panel problem. Where earlier cockpit fonts went monolinear and rigid, B612 keeps stroke contrast, opens counters, and lengthens ascenders and descenders. Each word’s silhouette resolves quickly. At stroke junctions it carries small notches (light traps) that keep joins from filling in on bright, low-contrast displays. The result has quietly human character quirks that are grounded in measured legibility gains rather than style. In 2017, B612 was released as open source through the Eclipse Polarsys project.

ABOUT PLANETAIRE MONO

By historical accident, B612 alone is not a usable document or application font. The versions in circulation, including the one on Google Fonts, have oddities and uneven symbols that make them awkward for use in modern applications and terminals. Planetaire Mono arises from this need. It merges B612's letters and digits into Hack Nerd Font's base:

- **B612 letterforms** for letters, digits, and extended Latin, Greek, and Cyrillic.
- **Hack punctuation and symbols** for {}[]()<> and the rest.
- **Ten variants across five weights** (400/500/600/700/800), including added SemiBold (600) and ExtraBold (800) weights, the latter especially useful as boldface in the terminal.
- **Over 10,000 Nerd Font icons** (Powerline, Font Awesome, Devicons) in the Extended package, part of 12,000+ total glyphs.
- **A dotted zero:** B612's zero with a center dot for clear 0 vs O, in circle (default) and rectangle (ss01) variants.

Hack has its own lineage: Bitstream Vera Mono (2003) became DejaVu Mono, which Chris Simpkins reworked in 2015 into a face tuned for source code at small sizes. Its signature is functional punctuation: brackets, braces, parentheses, and operators are drawn at a heavier weight and given extra spacing next to letters, so they stay distinct in dense code. B612's own punctuation lacks that weight, which is why Planetaire borrows these heavier symbols from Hack.

Planetaire Mono carries a new name for clarity and to comply with the B612 license.

A PERSONAL NOTE

Like architecture, typography is a functional art form. A design may initially appear attractive, but you can't know its true qualities until you live close to it. Or work within it.

A couple of years ago, I was building a terminal and surveyed monospace fonts to find the best ones. B612 wasn't my obvious first choice, but after trying many of the classic modern options available as Nerd Fonts, I came to realize it still just *felt* better over time. But I was disappointed with the technical flaws that made it hard to use as a full replacement for a modern, high-quality workhorse typeface such as Hack or JetBrains Mono. I made an adapted hybrid of B612 that I was quite happy with, but it wasn't in a clean enough form to publish.

Now, Claude Code and Opus 4.8 have made it a pleasure to consolidate this work as Planetaire Mono. I've used dozens of terminal fonts over the years, and it is now what I use every day.

Thanks to agentic coding, monospace fonts are now in greater use than anyone could ever have imagined. I hope Planetaire Mono's aesthetics lighten the hours you spend with your agents, editors, and terminals.

Planetaire Typeface Specification

DESIGN

Classification	Monospace (fixed)
Letterforms	B612 Mono (humanist)
Packages	Extended, Text
Styles	10 (5 weights × 2)
Units per em	2000
Advance width	1204 (0.602 em)
Italic angle	0°, true italics

WEIGHTS

Regular	400
Medium	500
SemiBold	600
Bold	700
ExtraBold	800

VERTICAL METRICS (per 2000 em)

Cap height	1520
x-height	1120
Ascender	1856
Descender	-472
Line gap	0

GLYPH COVERAGE

Extended	12,138 glyphs
Text	1,317 glyphs

FORMATS

Local install	TTF
Web	WOFF2 + @font-face CSS

OPENTYPE

default	circle-dot zero
ss01 / zero	rectangle-dot zero

LICENSE

All packages	SIL OFL 1.1

Planetaire Text Sample

ENGLISH

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump!

In the great void between stars, instruments must be read without error. Every glyph must be unambiguous: the digit 0 distinct from the letter O, the numeral 1 clearly not a lowercase l or uppercase I. B612 was born from this requirement. Originally designed for cockpit displays at Airbus, where a misread character could mean the difference between FL350 and FL850. Planetaire Mono inherits that precision and pairs it with the full symbol coverage a programmer needs: braces, brackets, pipes, arrows, and 12,000 icons ready for a modern terminal.

FRENCH · GERMAN · SPANISH · TURKISH

Les naïfs ægithales hâtifs pondent au zéphyr joyeux. Falsches Üben von Xylophonmusik quält jeden größeren Zwerg. El veloz murciélago hindú comía feliz cardillo y kiwi. Pijamalı hasta yağız şoföre çabucak güvendi.

GREEK

Ξεσκεπάζω τὴν ψυχοφθόρα βδελυγμία.

CYRILLIC

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

Planetaire Iconic Texts

ALAN TURING · “COMPUTING MACHINERY AND INTELLIGENCE” (1950)

I propose to consider the question, “Can machines think?” This should begin with definitions of the meaning of the terms “machine” and “think.” The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words “machine” and “think” are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, “Can machines think?” is to be sought in a statistical survey such as a Gallup poll. But this is absurd.

Network Working Group
Request for Comments: 1

Steve Crocker
UCLA
7 April 1969

Title: **Host Software**
Author: **Steve Crocker**
Installation: UCLA
Date: 7 April 1969
Network Working Group Request for Comment: 1

Introduction

The software for the ARPA Network exists partly in the IMPs and partly in the respective HOSTs. BB&N has specified the software of the IMPs and it is the responsibility of the HOST groups to agree on HOST software.

During the summer of 1968, representatives from the initial four sites met several times to discuss the HOST software and initial experiments on the network. There emerged from these meetings a working group of three, Steve Carr from Utah, Jeff Rulifson from SRI, and **Steve Crocker** of UCLA, who met during the fall and winter. The most recent meeting was in the last week of March in Utah. Also present was Bill Duvall of SRI who has recently started working with Jeff Rulifson.

Somewhat independently, Gerard DeLoche of UCLA has been working on the HOST-IMP interface.

I present here some of the tentative agreements reached and some of the open questions encountered. Very little of what is here is firm and reactions are expected.

I. A Summary of the IMP Software

Messages

Information is transmitted from HOST to HOST in bundles called messages. A message is any stream of not more than 8080 bits, together with its header. The header is 16 bits and contains the following information:

Destination	5 bits
Link	8 bits
Trace	1 bit
Spare	2 bits

The destination is the numerical code for the HOST to which the message should be sent. The trace bit signals the IMPs to record status information about the message and send the information back to the NMC (Network Measurement Center, i.e., UCLA). The spare bits are unused.

Planetaire Code Specimen: microGPT

Andrej Karpathy's microGPT: the scalar autograd engine at the core of a complete GPT training loop in 200 lines of pure Python.

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forward pass
        self.grad = 0                   # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad
```

Planetaire Terminal

```python
import math

def analyze_trajectory(altitude: float, velocity: float) -> dict:
    """Calculate orbital parameters."""

    G = 6.674e-11  # gravitational constant
    M = 5.972e24

    if altitude > 400_000:
        orbit_type = "LEO"
    elif altitude > 35_786_000:
        orbit_type = "GEO"

    period = 2 * math.pi * math.sqrt(altitude**3 / (G * M))

    return {"type": orbit_type, "period": period, "v": velocity}
```


```python
import math

def analyze_trajectory(altitude: float, velocity: float) -> dict:
    """Calculate orbital parameters."""

    G = 6.674e-11  # gravitational constant
    M = 5.972e24

    if altitude > 400_000:
        orbit_type = "LEO"
    elif altitude > 35_786_000:
        orbit_type = "GEO"

    period = 2 * math.pi * math.sqrt(altitude**3 / (G * M))

    return {"type": orbit_type, "period": period, "v": velocity}
```

```
planetaire $ eza -l --icons=always . ./docs/specimen/
.:
drwxr-xr-x@     - levy 15 Feb 23:07  devtools
drwxr-xr-x@     - levy 15 Feb 23:07  docs
drwxr-xr-x@     - levy 15 Feb 23:07  fonts
.rw-r--r--@  7.6k levy 16 Feb 09:14  LICENSE
.rw-r--r--@  1.3k levy 16 Feb 09:14  Makefile
.rw-r--r--@  6.2k levy 16 Feb 09:14  pyproject.toml
.rw-r--r--@  6.4k levy 15 Feb 23:07  README.md
drwxr-xr-x@     - levy 15 Feb 23:07  src
.rw-r--r--@   63k levy 16 Feb 09:14  uv.lock
./docs/specimen/:
.rw-r--r--@  188k levy 16 Feb 09:14  planetaire-mono-specimen.pdf
.rw-r--r--@  9.1k levy 15 Feb 23:07  planetaire-mono-specimen.typ

planetaire $ python -c "print('Hello from Planetaire Mono!')"
Hello from Planetaire Mono!

planetaire $ git log --oneline -3
5bd69c5 Switch B612 source to original polarsys/b612
a1c8e3f Add font comparison and regression detection
e927d01 Refactor merge pipeline for original B612
```


```
planetaire $ eza -l --icons=always . ./docs/specimen/
.:
drwxr-xr-x@     - levy 15 Feb 23:07  devtools
drwxr-xr-x@     - levy 15 Feb 23:07  docs
drwxr-xr-x@     - levy 15 Feb 23:07  fonts
.rw-r--r--@  7.6k levy 16 Feb 09:14  LICENSE
.rw-r--r--@  1.3k levy 16 Feb 09:14  Makefile
.rw-r--r--@  6.2k levy 16 Feb 09:14  pyproject.toml
.rw-r--r--@  6.4k levy 15 Feb 23:07  README.md
drwxr-xr-x@     - levy 15 Feb 23:07  src
.rw-r--r--@   63k levy 16 Feb 09:14  uv.lock
./docs/specimen/:
.rw-r--r--@  188k levy 16 Feb 09:14  planetaire-mono-specimen.pdf
.rw-r--r--@  9.1k levy 15 Feb 23:07  planetaire-mono-specimen.typ

planetaire $ python -c "print('Hello from Planetaire Mono!')"
Hello from Planetaire Mono!

planetaire $ git log --oneline -3
5bd69c5 Switch B612 source to original polarsys/b612
a1c8e3f Add font comparison and regression detection
e927d01 Refactor merge pipeline for original B612
```

Planetaire Character Set

BASIC LATIN UPPERCASE: from B612

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

BASIC LATIN LOWERCASE: from B612

a b c d e f g h i j k l m n o p q r s t u v w x y z

DIGITS: 0-9 from B612 (zero modified with center dot for O/0 disambiguation)

0 1 2 3 4 5 6 7 8 9

PUNCTUATION AND SYMBOLS: from Hack

! " # $ % & ' ( ) * + , - . / : ; < = > ? @ [\ ] ^ _
` { | } ~

EXTENDED LATIN: from B612

À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß

à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ÿ

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ď Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ě ě

Ğ ğ Ġ ġ Ģ ģ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Į į İ ı Ĳ ĳ Ĵ ĵ Ķ ķ

GREEK: from B612

Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

CYRILLIC: from B612

А Б В Г Д Е Ж З И Й К Л М Н О П Р С Т У Ф Х Ц Ч Ш Щ Ъ Ы Ь Э Ю Я

а б в г д е ж з и й к л м н о п р с т у ф х ц ч ш щ ъ ы ь э ю я

Planetaire Weight Comparison

REGULAR (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

ITALIC (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

MEDIUM (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

MEDIUM ITALIC (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

SEMIBOLD (600)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

SEMIBOLD ITALIC (600)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

BOLD (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

BOLD ITALIC (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

EXTRABOLD (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

EXTRABOLD ITALIC (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Planetaire Size Waterfall

Planetaire Mono from caption to display size, holding its proportions throughout.

8 Planetaire Mono

9 Planetaire Mono

10 Planetaire Mono

11 Planetaire Mono

12 Planetaire Mono

14 Planetaire Mono

16 Planetaire Mono

20 Planetaire Mono

24 Planetaire Mono

30 Planetaire Mono

36 Planetaire Mono

44 Planetaire Mono

LEGIBILITY AT DISPLAY SIZE

Il1| O0o 0123

Planetaire Legibility

CHARACTER DISAMBIGUATION

I l 1 |	uppercase I · lowercase l · digit 1 · pipe
O 0 o	uppercase O · digit 0 · lowercase o
r n m	r · n · m — clearly distinct in B612
5 S 8 B	digit 5 vs S · digit 8 vs B
2 Z 6 G	digit 2 vs Z · digit 6 vs G
; :	semicolon vs colon — distinct dot size and spacing
( ) { } [ ]	parens vs braces vs brackets
- – —	hyphen-minus vs en dash vs em dash
“ ” "	left double quote vs right double quote vs straight double quote
‘ ’ ' `	left single quote vs right single quote/ apostrophe vs straight quote vs backtick

ZERO DOT VARIANTS (OpenType)

Default (circle dot)

0 0 O o

FL350 FL850 10.0.0.1

ss01 / zero (rectangle dot)

0 0 O o

FL350 FL850 10.0.0.1

BRACKET AND DELIMITER PAIRS

( ) { } [ ] < > « » ‹ ›

MATHEMATICAL AND PROGRAMMING OPERATORS

+ − * / = ≠ ≤ ≥ ± × ÷ → ← ↑ ↓

Planetaire Symbols and Special Characters

STANDARD PUNCTUATION

. , ; : ! ? - - – ( ) [ ] { } / \ @ # $ % ^ & *
_ + = ~ `

QUOTES AND APOSTROPHES

" ' ` “ ” ‘ ’ « » ‹ ›

straight double · straight single · backtick · left double · right double · left single · right single/apostrophe · guillemets · single guillemets

TYPOGRAPHIC AND SPECIAL CHARACTERS

§ ¶ © ® ™ ° · … † ‡ ¤ ¢ £ ¥ € ¬ ¦

section · pilcrow · copyright · registered · trademark · degree · middle dot · ellipsis · dagger · double dagger · currency · cent · pound · yen · euro · not · broken bar

SLASHES AND STROKES

/ \ | ¦ ⁄ /

solidus · reverse solidus · vertical bar · broken bar · fraction slash · division slash

BOX DRAWING AND BLOCK ELEMENTS

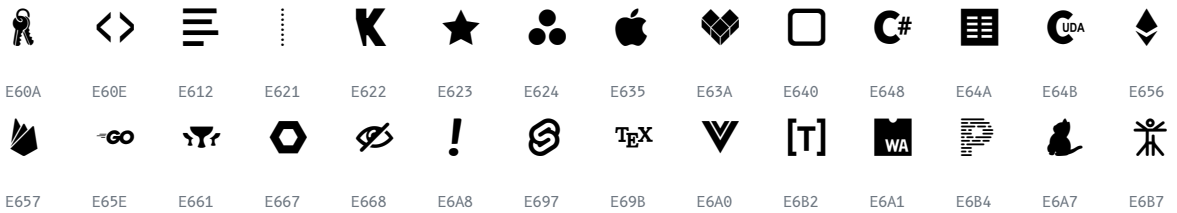
─ │ ┌ ┐ └ ┘ ├ ┤ ┬ ┴ ┼ ═ ║ ╔ ╗ ╚ ╝ ╠ ╣ ╦ ╩ ╬ ▀ ▄ ▌ ▐ ▐
░ ▒ ▓

Nerd Font Icons

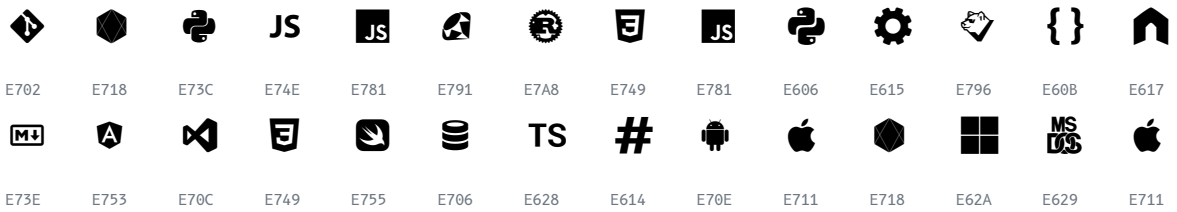
A selection of the 10,000+ Nerd Font icons included via Hack Nerd Font. Each icon shown with its Unicode codepoint.

UI AND COMMON

FILE TYPES



DEVELOPMENT



WEATHER

POWERLINE

Planetaire Provenance

Credits

Planetaire Mono	Assembled and maintained by Joshua Levy.
B612 Mono	Letterforms by Intactile Design for Airbus. Planetaire Mono takes its letters, digits, and extended Latin, Greek, and Cyrillic glyphs from B612.
Hack	Chris Simpkins' typeface designed for source code. It provides Planetaire Mono's base font structure: punctuation, symbols, metrics, and Nerd Font integration.
Nerd Fonts	Ryan McIntyre's icon patching project. Planetaire Mono Extended includes 10,000+ developer icons, including Powerline, Font Awesome, Devicons, Material Design, and more, via the Hack Nerd Font base.
carlosedp	Carlos Eduardo de Paula's B612 Nerd Font fork, which inspired the dotted zero. It is not a build dependency.

Build Pipeline

Planetaire Mono is built with a custom Python pipeline using fontTools. For each weight variant, the pipeline loads Hack Nerd Font as the base, merges B612's letter and digit glyphs by Unicode range (normalizing units per em from 2048 to 2000), adds a center dot to B612's zero for O/0 disambiguation, sets family and version metadata, applies post-processing fixes, and validates coverage and style linking. The intermediate B612 weights (Medium, SemiBold, and ExtraBold) are generated from its base weights with FontForge emboldening before merging.

Two Packages: Text and Extended

Both packages share the same letterforms and the same 10 variants, and both ship in two formats: TTF for local install and WOFF2 (with a ready `@font-face` stylesheet) for the web. They differ only in glyph coverage:

- **Planetaire Mono Extended** is the full font: everything in Text plus the 10,000+ Nerd Font icons and Powerline glyphs that terminals and CLIs draw, so it is a superset of Text. Recommended for local and terminal use, where TTF is the standard option.
- **Planetaire Mono Text** is a lightweight subset (no icons), so it is far smaller, and its web CSS now uses Regular/Bold Latin, Greek, and Cyrillic WOFF2 slices with optional Regular/Bold italics. Recommended for the web, where the WOFF2 stylesheet is the standard option.

Either package works for either purpose; the recommendations are just the common, size-conscious defaults.

Planetaire Mono Text

```
The quick brown fox jumps over the lazy dog
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz
0123456789 !@#$%^&*()[]{} <>=+ — Il1| O0o
Greek ΑΒΓ αβγ · Cyrillic АБВ абв · Box ┌┬┐ │ ├┼┤ └┴┘ ▓▒░
```

Spacing Review

Planetaire Mono is built to a single cell width: every glyph (and intentional double-width glyphs at exactly 2x) shares one advance. B612's letters and the FontForge-emboldened weights are normalized to that cell, recentered, and condensed only where ink would otherwise bleed. The two panels below are the visual proof. Review them to confirm no glyph is trimmed and all weights align.

STANDARD CODING CHARACTERS: TRUE CELL WIDTHS

Red rules mark each glyph's advance. Equal-width cells with ink inside mean a clean monospace grid, nothing trimmed.

```
ABCDEFGHIJKLMNOPQRSTU
VWXYZ

abcdefghijklmnopqrstu
vwxyz

0123456789

!"#$%&'()*+,-./:;<=>?@

[\]^_`{|}~
```

Cell-filling glyphs (box-drawing, powerline) reach the rules by design:

```
─│┌┐└┘├┤┬┴┼ ▒ ▓ ▒ ▒ ▶◀
```

STANDARD CODING CHARACTERS: TRUE CELL WIDTHS (ITALIC)

Red rules mark each glyph's advance. Equal-width cells with ink inside mean a clean monospace grid, nothing trimmed.

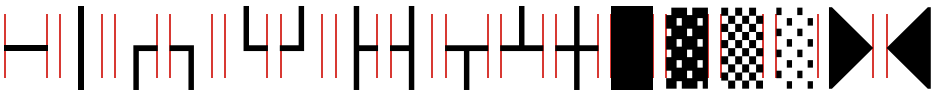
```
ABCDEFGHIJKLMNOPQRSTU
VWXYZ

abcdefghijklmnopqrstu
vwxyz
```

0 1 2 3 4 5 6 7 8 9

! " # $ % & ' ( ) * + , - . / : ; < = > ? @

[\ ] ^ _ ` { | } ~

Cell-filling glyphs (box-drawing, powerline) reach the rules by design:



WEIGHT ALIGNMENT: EVERY WEIGHT IS THE SAME WIDTH

The same string in all ten variants; the red rule marks each line's right edge. A single vertical line means identical width across every weight.

REGULAR (400)	`if (x == 0) { i = l1|O; }`
ITALIC (400)	*`if (x == 0) { i = l1|O; }`*
MEDIUM (500)	`if (x == 0) { i = l1|O; }`
MEDIUM ITALIC (500)	*`if (x == 0) { i = l1|O; }`*
SEMIBOLD (600)	`if (x == 0) { i = l1|O; }`
SEMIBOLD ITALIC (600)	*`if (x == 0) { i = l1|O; }`*
BOLD (700)	`if (x == 0) { i = l1|O; }`
BOLD ITALIC (700)	*`if (x == 0) { i = l1|O; }`*
EXTRABOLD (800)	`if (x == 0) { i = l1|O; }`
EXTRABOLD ITALIC (800)	*`if (x == 0) { i = l1|O; }`*

LICENSE

Planetaire Mono is released under the **SIL Open Font License 1.1** (OFL-1.1), the standard license for open fonts. In practical terms:

- **Use it for anything, free.** Set text in Planetaire Mono in documents, books, websites, apps, videos, and commercial products, with no fee and no permission needed.
- **No credit required for use.** Setting text in the font does not obligate you to attribute Planetaire Mono or B612 anywhere, including in publications, commercial work, or open source software. Credit is welcome but optional.
- **Bundle and redistribute freely, but keep the license with the files.** If you ship the font files themselves (embedding web fonts, packaging them in an app or OS), include the bundled license and copyright notices alongside them. You may not sell the font files on their own.
- **Don't reuse the B612 name for modified versions.** OFL lets an author reserve font names, and "B612" is reserved, so a derivative cannot be distributed under that name.

This is a summary, not legal advice; the full OFL text is binding. The upstream sources carry their own licenses: **B612** under OFL-1.1 and EPL-2.0; **Hack** under MIT and the Bitstream Vera License (Hack's symbols descend from Bitstream Vera, which reserves the names "Bitstream" and "Vera"); **Nerd Fonts** glyph fonts under OFL-1.1 with the patcher tooling under MIT. Full license texts for all of these ship in the licenses/ folder of each release archive. The build tooling in this repo is MIT.